SPAC: Session-Bound Path Authentication for Distributed Neural Inference

Zhengzhe Xiang, Yinlin Chen, Guanjie Cheng, Javid Taheri, Shuiguang Deng

Abstract—Distributed neural inference executes ordered model shards across independently managed devices, creating a need to authenticate session participation and activation transfers. We propose SPAC (Session-bound Path Authentication Chain), which binds device identities, ordered hop records, registered shard claims, and cross-hop encoded-activation continuity into one proof. In a corruption game where at most k of N devices may be compromised ($k < N$), a reduction to message authentication code (MAC) unforgeability bounds fresh false attribution of honest-device participation. At the claim-binding tier, SPAC authenticates registered claims and encoded transfers but does not provide device-side hardware attestation or prove the correctness of every inference step; hardware-endorsed measurements require additional evidence. No accepted forgery was observed in the tested proxy, topology, and digest-inconsistent injection workloads. SPAC adds 0.65–2.56 ms in the evaluated multilayer perceptron (MLP) and ResNet configurations, with proof metadata growing linearly in the chain length. The encoding study exposes successful within-cell substitutions under float forwarding. Authenticating transmitted integer codes removes this substitution mechanism on those links; the corresponding ResNet experiment observed an accuracy difference of 0.01 percentage points. These results distinguish authenticated path records and encoded transfer consistency from the semantic correctness of compromised computation.

Index Terms—Distributed inference, path-participation authentication, cross-hop activation continuity, session binding, trust-boundary attestation, neural network security

1 Introduction

Distributed neural inference partitions a model into ordered shards that execute on different devices. Edge–cloud systems use this structure to place computation across heterogeneous resources and reduce inference latency [1], [2]. The setting differs from federated learning, which aggregates updates from decentralized training data [3], and from prediction-serving systems that coordinate model deployment and requests [4]. Here, one inference request traverses a designated device chain, and intermediate activations connect the local computations.

Consider a model owner who assigns one shard to each independently managed participant, avoiding delivery of the complete model to any single participant. The owner registers a route, device identities, and claims about the assigned shards. After an inference session, the owner needs evidence linking the returned result to those registered participants and their activation transfers. This audit requirement arises even when the final prediction is accurate: a functionally equivalent proxy may produce an acceptable output while bypassing the designated route. Split deployment motivates the participation policy; authenticating that policy does not itself establish model confidentiality or prevent model extraction.

An adversary may exploit this gap by substituting a proxy, skipping a device, reordering authentication records, or changing an activation in transit. Model-extraction results provide concrete constructions of functionally equivalent proxies under their respective assumptions [5], [6]. Replayed or spliced records pose a related problem: individually valid credentials or tags need not describe the current inference session. The verifier therefore needs a session-specific account of identities, record order, registered claims, and adjacent activation transfers.

This paper studies *path-participation authentication*. Under the baseline trust tier, an accepted proof authenticates honest-device participation in the security game of section 3 and consistency with registered declarations and encoded transfers. Hardware-endorsed code/model measurements require additional evidence. Computational correctness is a complementary objective: a proxy may compute correctly without following the designated route, while a compromised participant may authenticate a self-consistent but incorrect computation. The distinction determines both the protocol's security target and how its evaluation must be interpreted.

Three requirements shape the design. *First, output agreement must be separated from participation evidence.* A check such as $\|y - y_{\mathrm{ref}}\|_p \leq \delta$ evaluates numerical agreement, not the identities or transfers that produced the result. A proxy satisfying the same predicate is equally acceptable to this check. Device credentials supply identity evidence, but a path proof must bind their use to the current session and designated route.

Second, local evidence must be linked across hops. Collective remote attestation provides evidence about device state [7], [8]. State reports alone do not relate one hop's session output to the next hop's received input. Message authentication codes (MACs) and signatures can carry these fields; the design question is how the verifier combines them into one ordered proof and checks the dependencies. Distributed flow-attestation predecessors address related dependencies, making the precise authenticated object and trust assumptions central to the comparison in section 2.

Third, the authenticated object must be explicit. Heteroge-

neous devices can use different numeric representations. A precision conversion before authentication may change an activation's encoding, whereas forwarding the same serialized bytes preserves their identity. Under float forwarding, comparing lossy encodings can also leave distinct tensors indistinguishable within one quantization cell. Authenticating transmitted integer codes instead binds the wire object before decoding. These choices affect compatibility and residual substitution risk, so they form part of the protocol contract rather than an implicit implementation detail.

We propose SPAC (Session-bound Path Authentication Chain) to meet these requirements for linear-chain inference. Each hop authenticates its identity, registered claim, input/output digests, and predecessor state under a fresh session context. The verifier checks the registered route, the tags and chain state, and the adjacent constraint $v_{i-1} = u_i$. This produces one session-consistent encoded transcript without replaying inference and supports both float forwarding and transmitted-code authentication. The construction, security game, and encoding contract make each claimed guarantee's trust boundary explicit.

The contributions are as follows:

1) **Session-bound path authentication.** We construct a proof combining identities, ordered authentication records, registered shard claims, and cross-hop encoded-activation continuity. We formalize its trust tiers and $k < N$ corruption game, and bound fresh false attribution of honest participation by a context-freshness term and a multi-user MAC-forgery term (sections 3 and 4). The game excludes honest-node signing in the challenge session.
2) **An explicit activation-transfer contract.** We distinguish the objects authenticated by float forwarding and transmitted-code authentication, including their relation to numeric conversion and returned-output encoding. We analyze compatibility and quantify within-cell substitutions and boundary sensitivity. This connects the abstract digest constraint to the tensors and bytes used by a deployment (sections 4.1 and 5.4).
3) **Evaluation tied to the security claims.** We test path and topology attacks, isolate the contribution of individual bindings, study encoding compatibility and residual risk, and measure system cost. The MLP, ResNet, and ViT experiments distinguish rejected digest-inconsistent attacks from accepted within-cell substitutions. The primary MLP/ResNet grid incurs 0.65–2.56 ms of added latency with linear proof metadata; transmitted-code mitigation is evaluated separately (section 5).

Section 2 positions SPAC against prior work. Sections 3 and 4 define its assumptions, protocol, and analysis. Section 5 answers the evaluation questions arising from the three contributions, and section 6 concludes the paper.

2 Related Work

The relevant literature addresses three connected questions: how inference is distributed, what evidence authenticates distributed execution, and how stronger computation or confidentiality guarantees can be added. We organize the comparison around these proof targets and SPAC's position among them.

2.1 Distributed Inference and Proxy Execution

DDNN [1] maps neural computation across end devices, edge nodes, and the cloud. Edgent [2] combines partitioning and early exit for device–edge inference. These systems motivate the sequential shard abstraction in definition 1: each device consumes an activation produced upstream and passes its output downstream. Their placement and performance objectives establish the deployment setting; the participation policy studied here asks whether the registered route has authenticated the transfers of one session. Branching and dynamic routing require additional rules beyond this base chain.

Model extraction makes proxy execution a concrete threat to output-based checks. Carlini et al. [5] study cryptanalytic extraction of ReLU networks; subsequent soft-label [6] and hard-label [9], [10] results extend the available constructions under different assumptions. These results motivate the possibility of an acceptable proxy output, rather than imply that arbitrary architectures are universally extractable.

2.2 Authentication, State Attestation, and Distributed Flow Attestation

Tamper-evident logging [11] and sequential aggregate signatures [12] provide foundations for authenticating messages and ordered records. A protocol using these primitives must specify which session, identity, position, and payload fields are bound, and which dependencies its verifier checks. SPAC uses per-device hash-based message authentication code (HMAC) tags and a predecessor state, with adjacent digest equality as a separate acceptance condition.

Collective remote attestation (CRA) addresses evidence about populations of devices. SANA [7] and ESDRA [13] study scalable device attestation; Catt [8] organizes CRA properties and trust assumptions, and PIRANHAS [14] addresses privacy in asynchronous collectives. RADIS [15] considers distributed IoT services, bringing the attestation target closer to an application spanning multiple participants. This literature motivates the distinction between registered-state evidence and evidence linking the data transfers of one inference session. Our CRA-style baseline abstracts state/registration checking without an activation-continuity clause.

Control-flow attestation records dynamic program behavior. C-FLAT [16], LO-FAT [17], SpecCFA [18], and TRACES [19] develop runtime evidence and associated hardware or trusted-execution support. ARCADIS [20] is especially relevant: it accumulates service input/output information and local flow evidence, with signatures and logical clocks connecting asynchronous interactions. Cross-service dependencies therefore have direct precedents. SPAC's focus is their integration with a session-bound inference transcript and an explicit activation-encoding contract. At its claim-binding tier, SPAC authenticates declarations with protected verifier keys and secret honest-device keys; runtime measurement evidence is an optional additional tier, not an implicit consequence of the MAC chain.

2.3 Verifiable Inference and Confidential Execution

Verifiable inference constrains the computation performed on a declared model. ZKML [21], Artemis [22], and zkLLM [23] develop proof systems for ML inference; SVIP [24], VeriLLM [25], TensorCommitments [26], and Anchuri et al. [27] explore different public-verifiability and cost tradeoffs. These methods address an assurance dimension complementary to specified-device participation. They motivate the optional correctness composition in section 4.5.

Trusted execution and confidential computing provide isolation and measurement mechanisms. Chrapek et al. [28] evaluate CPU/GPU TEE costs, while Careful Whisper [29], Trust Alchemist [30], and AGATE [31] study attestation and its composition across workloads or participants. Secure-inference systems such as ENSI [32] and EncFormer [33] pursue protected computation. For SPAC, independently verifiable measurement evidence supplies the optional TA-4 endorsement; the baseline does not inherit every assurance of these systems merely by binding a registered measurement.

2.4 Positioning of SPAC

SPAC combines session-bound identities, ordered records, registered claims, and adjacent encoded-activation digests in one proof for a fixed inference chain. Its contribution lies in this construction and in the explicit relationship between authenticated bytes, activation encodings, and trust tiers. The resulting guarantee is intentionally narrower than verifiable inference or hardware attestation: it authenticates path participation and encoded-transfer consistency, while computational correctness and hardware-backed runtime integrity require additional mechanisms. This positioning determines the scope of the experiments in section 5.

3 System Model and Threat Model

3.1 System Model

Definition 1 (Linear-Chain Distributed Inference System)**.** *A model f_Θ is partitioned into N ordered shards $\Theta = (\theta_1, \ldots, \theta_N)$, with shard θ_i assigned to device D_i. Honest execution follows*

$$a_i = F_i(\theta_i, a_{i-1}), \quad i = 1, \ldots, N, \tag{1}$$

where $a_0 = x_{\text{task}}$ and $a_N = y$. Each nonterminal device forwards its output to its unique successor. The chain has no branching, merging, or conditional routing; intra-shard structures, including residual connections, are encapsulated in F_i.

In Fig. 1, an offline *Enrollment Authority* (EA) enrolls devices and distributes keys. The model owner acts as (or delegates) the *verifier* $\mathcal{V}$: it registers each device's identity and key together with the claim digest μ_i of its assigned shard through the EA, and after each session checks that the proof P is consistent with each device D_i's registered identity, claim, and position. At the claim-binding baseline this establishes only consistency with the registered declaration, not that D_i actually executed the declared θ_i; that association additionally requires hardware evidence. The owner's question is therefore not "was the computation correct?" but "is this session proof consistent with my designated device path and registered claims?"

The roles are:

- **Enrollment Authority (EA):** performs device enrollment, key distribution, and claim registration. The EA acts as an offline trusted third party and is not involved in online inference or verification.
- **Verifier ($\mathcal{V}$):** holds the registration database $\mathcal{R} = \{(i, ID_i, \mu_i, K_i)\}_{i=1}^{N}$, recording each device's index, identity, claim digest, and key. The verifier generates fresh session identifiers, receives the inference proof P, and runs the verification algorithm.
- **Inference devices $D_1, \ldots, D_N$:** each device holds a key K_i, a model shard θ_i, and a local execution environment; device D_i occupies the fixed i-th chain position and constructs the hop tuple Π_i (section 4).

3.2 Trust Assumptions

We make the following assumptions.

TA-1: The integrity of the verifier's registration database and key storage is protected by a trusted execution environment (TEE), hardware security module (HSM), or comparable root-of-trust mechanism, and the adversary cannot directly read K_i from honest devices.

TA-2: Enrollment is performed over a secure channel, so the establishment of (ID_i, μ_i, K_i) is not subject to man-in-the-middle tampering.

TA-3 (claim-binding baseline): Without a TEE, measured boot, or secure boot, μ_i is the registered digest of the device's declared shard metadata; successful verification means "consistent with the registered declaration," not that the device currently runs that code/model.

TA-4 (optional strengthening): To associate a registered claim with a hardware-endorsed code/model measurement, measured boot, secure boot, or a TEE quote must additionally bind the declared shard metadata to the hardware identity, and the verifier must validate this evidence at enrollment or session start.

The enrollment assumptions are:

EA-1 (cryptographic primitives): The hash function H and HMAC are modeled under the random-oracle heuristic and standard cryptographic assumptions.

EA-2 (trusted enrollment): The enrollment phase is trusted.

Verifier-side TEE/HSM protection and secure enrollment are standard assumptions in remote-attestation schemes. TA-3 is the minimum deployable baseline without a device-side hardware root of trust; TA-4 is the upgrade path associating μ_i with a hardware-endorsed measurement. Neither tier verifies every runtime computation; the boundary is examined in section 5.6. HMAC-SHA256's EUF-CMA security underpins theorem 1.

3.3 Adversary Model

The adversary $\mathcal{A}$ is a probabilistic polynomial-time (PPT) algorithm with the following capabilities:

1) $\mathcal{A}$ may adaptively corrupt up to $k < N$ devices, obtaining their keys and model shards.

2) $\mathcal{A}$ fully controls network communication and may eavesdrop, tamper, replay, or launch man-in-the-middle attacks.

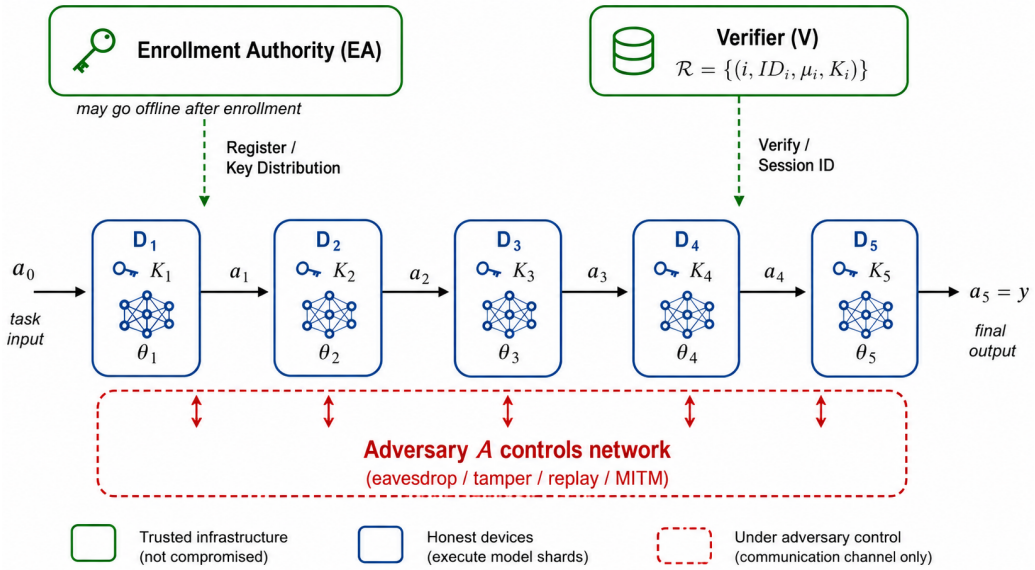


Fig. 1. System and threat model (N=5). The adversary controls inter-device communication (red region) and may corrupt at most $k < N$ devices. Enrollment and verification are trusted.

3) $\mathcal{A}$ may launch model-extraction attacks, including hard-label settings, to build a functionally equivalent proxy model.

4) $\mathcal{A}$ may cause compromised devices to emit arbitrary activations, inference results, and tags consistent with their own keys; the base scheme does not claim such activations are semantically correct.

5) $\mathcal{A}$ cannot corrupt the verifier or the EA, and cannot directly read keys from honest devices (TA-1).

3.4 Participation Definition and Security Game

Definition 2 (Actual Participation)**.** *An honest device D_i participates in session sid by receiving a_{i-1}, computing $a_i = F_i(\theta_i, a_{i-1})$, constructing the digests, binding message, and tag (u_i, v_i, m_i, t_i), and outputting t_i according to section 4.*

The k-path participation authentication (PPA) experiment $\mathsf{PPA\text{-}Exp}^{k}_{\mathcal{A},N}(\lambda)$ proceeds as follows:

1) **Initialization.** $\mathsf{Setup}(1^\lambda, N, \mathrm{route_id})$ generates the device keys, claim digests, and registration database $\mathcal{R}$.
2) **Adaptive queries.** $\mathcal{O}_{\mathrm{Session}}(x_{\mathrm{task}})$ returns a fresh context ctx containing sid, nonce ν, expiry t_{exp}, route_id, model_ver, and $x_{\mathrm{hash}} = H(E_x(x_{\mathrm{task}}))$. $\mathcal{O}_{\mathrm{Sign}}(i, m)$ returns $\mathrm{HMAC}_{K_i}(m)$ for non-challenge sessions only. $\mathcal{O}_{\mathrm{Corrupt}}(i)$ reveals K_i and adds i to $\mathcal{C}$, subject to $|\mathcal{C}| \le k$.
3) **Challenge and forgery.** $\mathcal{A}$ chooses x^*_{task} and obtains a fresh context ctx^* with identifier sid^*. Honest-node signing queries for sid^* are forbidden; corrupted nodes can generate their own tags. The adversary outputs $P^* = (y^*, z^*_N, \Pi^*_1, \ldots, \Pi^*_N)$.

The experiment records honest signing events in a session log; this log operationalizes the actual-participation definition used by the reduction. The adversary wins if $\mathsf{Verify}(P^*, ctx^*, \mathcal{R}) = \mathsf{Accept}$ while some device $j \in \mathcal{U} = [N] \setminus \mathcal{C}$ did not actually participate in sid^*; $\mathcal{C}$ is the final corruption set. The goal is negligible advantage

$$\mathrm{Adv}^{k\text{-PPA}}_{\mathcal{A}}(\lambda) = \Pr[\mathsf{PPA\text{-}Exp}^{k}_{\mathcal{A},N}(\lambda) = 1] \tag{2}$$

for every PPT adversary and $k < N$. Theorem 1 analyzes this experiment.

3.5 Security Guarantees and Scope

Under the above assumptions, the base scheme provides the following guarantees.

PPA-1 (unforgeable participation tags): If verification accepts, the tag t_i of each honest node is a valid authenticator on its binding message m_i (eq. (7)); for a non-participating honest node to appear in the proof, the adversary must forge an honest node's tag or reuse a historical session tag.

PPA-2 (digest-chain consistency): If verification accepts, the proof satisfies $u_1 = H(E_x(x_{\mathrm{task}}))$, $v_{i-1} = u_i$ $(i \ge 2)$, $z_i = H(z_{i-1} \| t_i)$, and $H(E_a(y)) = v_N$: it records one end-to-end consistent digest chain rather than unrelated local tags.

PPA-3 (session freshness and replay resistance): Each sid binds a fresh nonce and an expiration time, so historical proofs cannot be replayed into new sessions. This does not by itself enforce single acceptance within an unchanged, unexpired session.

Under TA-3 alone, the μ_i check in PPA-1 means "consistent with the registered declaration"; only under TA-4 may μ_i be further read as hardware-endorsed code/model integrity evidence.

The base scheme does *not* cover:

OOS-1 (computational correctness verification): the scheme does not prove each hop computed correctly—only who participated, along which path, and that the digest chain is self-consistent.

OOS-2 (runtime code/model integrity without a hardware root of trust): without a hardware root of trust, μ_i

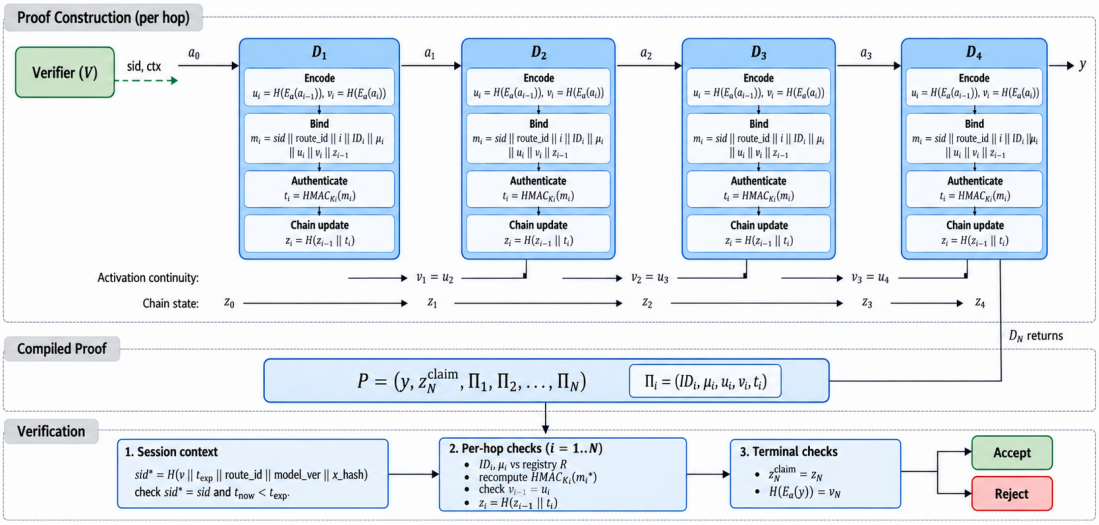


Fig. 2. SPAC overview. Each hop authenticates its identity, registered claim, activation digests, and predecessor state. The verifier checks these records, adjacent digest continuity, and the terminal output binding.

provides only claim binding, not proof that the device currently runs the enrolled code and model.

OOS-3 (side-channel protection): if K_i resides only in conventional software, side-channel leakage remains possible; stronger hardening uses a TEE, secure element, or HSM.

OOS-4 (semantic content safety): the protocol does not judge whether outputs are safe, useful, or policy-compliant, nor prevent compromised nodes from generating semantically incorrect yet locally self-consistent activations.

OOS-5 (verifier compromise): security requires an uncompromised verifier (TA-1); otherwise the adversary obtains all device keys and can forge arbitrary proofs. The asymmetric-signature extension in section 4.5 removes device signing keys from the verifier but does not establish runtime integrity.

Digest consistency concerns encoded objects: lossy encoding admits distinct raw tensors with identical codes. Within-cell substitution and near-boundary false rejection depend on the encoding configuration and are evaluated in section 5. Extensions and trust-boundary controls appear in sections 4.5 and 5.6.

4 Protocol Design

SPAC authenticates a session's ordered hop records and encoded activation continuity under the model of section 3. Fig. 2 shows its construction and verification.

4.1 Authentication and Encoding

Each enrolled device shares K_i with the verifier and uses HMAC-SHA256, a keyed-hash message authentication code. This inexpensive symmetric design requires protected verifier-side keys (TA-1); public verification requires signatures (section 4.5). Concatenation $\|$ assumes an agreed, unambiguous serialization of fields. Enrollment fixes canonical encoders E_x, E_a, and E_θ for inputs, activations, and model shards, including tensor metadata and encoding parameters.

TABLE 1
Authenticated objects in the forwarding configurations

	Float forwarding	Transmitted codes
Wire object	Float tensor a_i	Codes $w_i = E_a(a_i)$
Digest	$H(E_a(a_i^{\text{recv}}))$	$H(w_i^{\text{recv}})$, before decoding or casting
Hop input	Received float	Decoded codes
Link risk	Same-cell changes may pass; casts can change the encoding	Exact code-byte binding; no same-cell substitution of transmitted bytes

Table 1 distinguishes the two forwarding configurations. Both use the registered encoding and metadata; they differ in the object that the receiver authenticates.

In float forwarding, the consumer encodes the object it actually receives; preprocessing before encoding affects acceptance. QMatch on an independently recomputed or differently cast copy is a separate diagnostic. In code forwarding, the receiver hashes the actual bytes before conversion. Neither configuration proves compromised-hop semantics, and the returned y remains bound only through $E_a(y)$. Section 5.4 evaluates compatibility and residual risks.

Remark 1 (Encoding-match reference)**.** *For M coordinates quantized uniformly over $[-R, R]$ with $\Delta = 2R/2^b$, uniform quantization phase (SA-1), independent coordinates (SA-2), and coordinate drift at most $\epsilon_i < \Delta/2$ give the reference*

$$\Pr[E_a(\tilde{a}) = E_a(a)] \geq (1 - 2\epsilon_i/\Delta)^M. \tag{3}$$

Sparsity and correlation violate these assumptions in practice; this is not a deployment guarantee. Measured QMatch and rejection rates are reported in section 5.4.

4.2 Session and Hop Construction

Enrollment records the claim digest

$$\mu_i = H(\text{route_id} \| i \| E_\theta(\theta_i) \| h_i^{\text{code}} \| q_i), \tag{4}$$

where h_i^{code} identifies the code image and q_i its encoding configuration. Registration commits these fields at route position i; ID_i is authenticated in m_i. The verifier checks

this claim under TA-3; hardware endorsement additionally requires TA-4.

For fresh nonce ν, expiry t_{exp}, and task x_{task}, the verifier issues

$$sid = H(\nu \| t_{\text{exp}} \| \text{route_id} \| \text{model_ver} \| x_{\text{hash}}), \tag{5}$$

with $x_{\text{hash}} = H(E_x(x_{\text{task}}))$. The context ctx retains these fields and sid. Initialize $z_0 = H(sid \| \text{route_id} \| x_{\text{hash}})$.

After receiving a_{i-1}, honest device D_i computes $a_i = F_i(\theta_i, a_{i-1})$. In float forwarding,

$$u_i = \begin{cases} x_{\text{hash}}, & i = 1, \\ H(E_a(a_{i-1})), & i > 1, \end{cases} \qquad v_i = H(E_a(a_i)). \tag{6}$$

In transmitted-code authentication, $u_1 = x_{\text{hash}}$, $u_i = H(w_{i-1})$ for $i > 1$, and $v_i = H(w_i)$ use the actual received and transmitted codes. Each consumer computes on the decoded input. The returned y remains authenticated at the granularity of its registered output encoding. Each hop constructs

$$m_i = sid \| \text{route_id} \| i \| ID_i \| \mu_i \| u_i \| v_i \| z_{i-1}, \tag{7}$$

then computes

$$t_i = \text{HMAC}_{K_i}(m_i), \tag{8}$$

$$z_i = H(z_{i-1} \| t_i). \tag{9}$$

It appends $\Pi_i = (ID_i, \mu_i, u_i, v_i, t_i)$ and forwards the updated state. The tail returns $P = (y, z_N^{\text{claim}}, \Pi_1, \ldots, \Pi_N)$. Verification requires $O(N)$ tags and hashes, with $O(N\lambda)$ proof metadata, and does not replay inference.

Three-hop example. For $D_1 \to D_2 \to D_3$ with code forwarding, the verifier issues ctx for input a_0. Device D_1 sets $u_1 = x_{\text{hash}}$, computes a_1 and w_1, and produces (v_1, t_1, z_1) using eqs. (6) to (9). Device D_2 hashes the received w_1 into u_2, computes on its decoding, and extends the record and chain state. Device D_3 repeats this process and returns $P = (y, z_3^{\text{claim}}, \Pi_1, \Pi_2, \Pi_3)$. The verifier checks the tags and states, $v_1 = u_2$, $v_2 = u_3$, and the encoding of y. These equalities link the records of one session; checking them is the verifier's responsibility.

4.3 Verification

Algorithm 1 checks the session context, registered claims, tags, and continuity. Input pinning prevents task substitution; $v_{i-1} = u_i$ enforces adjacent encoded continuity; predecessor binding links the authenticated records. Expiry and fresh session binding prevent expired and cross-session proofs, but do not enforce single acceptance within a session.

4.4 Security Analysis

Observation 1 (Output-only verification)**.** *An output predicate cannot authenticate the executor: a proxy producing the same accepted output is indistinguishable to a verifier observing only that output.*

Algorithm 1 SPAC verification: Verify$(P, ctx, \mathcal{R})$

Require: Proof P, complete context ctx, registry $\mathcal{R}$
1: Reject if sid disagrees with eq. (5), $t_{\text{now}} \geq t_{\text{exp}}$, or $\#\text{tuples}(P) \neq N$
2: $z_0 \leftarrow H(sid \| \text{route_id} \| x_{\text{hash}})$
3: **for** $i = 1$ **to** N **do**
4: Parse $\Pi_i = (ID_i, \mu_i, u_i, v_i, t_i)$
5: Reject if $(ID_i, \mu_i) \neq (\mathcal{R}[i].ID, \mathcal{R}[i].\mu)$
6: Reject if $i = 1$ and $u_i \neq x_{\text{hash}}$
7: Reconstruct m_i using eq. (7) and z_{i-1}
8: Reject if $\text{HMAC}_{\mathcal{R}[i].K}(m_i) \neq t_i$
9: Reject if $i > 1$ and $v_{i-1} \neq u_i$
10: $z_i \leftarrow H(z_{i-1} \| t_i)$
11: **end for**
12: Reject if $z_N^{\text{claim}} \neq z_N$ or $H(E_a(y)) \neq v_N$
13: **return** Accept

Proposition 1 (Proxy acceptance)**.** *If extraction yields $\hat{f}$ with $\Pr_x[\|\hat{f}(x) - f_\Theta(x)\|_p \leq \delta] \geq 1 - \varepsilon_{\text{ext}}$, its output-only acceptance rate is at least $1 - \varepsilon_{\text{ext}}$. ReLU extraction provides relevant constructions [6]; their assumptions do not imply universal extractability of arbitrary architectures.*

Security game. The proof follows the k-PPA experiment of section 3: the session, corruption, and signing oracles, the corruption budget $|\mathcal{C}| \leq k$, the challenge context ctx^*, and the final honest set $\mathcal{U} = [N] \backslash \mathcal{C}$ are defined there. The session log records honest signing events; if the proof claims the i-th hop but no honest signing event consistent with sid exists, that hop is not actually participating in the session.

Theorem 1 (k-collusion robustness)**.** *Assume that the underlying MAC is EUF-CMA secure, the session nonce ν is sampled uniformly from $\{0,1\}^n$ (recommended $n \geq 128$), and the hash output length is η. For any PPT adversary $\mathcal{A}$ that corrupts at most $k < N$ devices, issues at most q_s signing-oracle queries, and requests at most q_{sid} sessions,*

$$\text{Adv}_{\mathcal{A}}^{k\text{-PPA}}(\lambda) \leq \frac{q_{\text{sid}}(q_{\text{sid}} - 1)}{2^{n+1}} + \frac{q_{\text{sid}}(q_{\text{sid}} - 1)}{2^{\eta+1}} + (N - k) \cdot \max_{j \in \mathcal{U}} \text{Adv}_{\mathcal{B}_j}^{\text{EUF-CMA}}(\lambda), \tag{10}$$

where $\mathcal{U} = [N] \setminus \mathcal{C}$ is the final set of honest devices and $\mathcal{B}_j$ makes at most q_s signing queries.

Proof. Partition the adversary's success event into three disjoint parts: E_ν, the challenge session reused a previously issued nonce; E_{sid}, two distinct complete session contexts with different nonces hashed to the same sid; and E_{fresh}, neither collision occurred, yet the adversary output an acceptable proof containing at least one honest device that did not actually participate.

Step 1: Bound the nonce-collision term. Since the session nonce is an n-bit uniform string and the adversary requests at most q_{sid} sessions, the birthday bound gives

$$\Pr[E_\nu] \leq \frac{q_{\text{sid}}(q_{\text{sid}} - 1)}{2^{n+1}}. \tag{11}$$

The subsequent argument conditions on $\neg E_\nu$.

Step 2: Bound the session-identifier collision term. Given distinct nonces, two different session contexts can

yield the same sid only through a hash collision on different inputs, so

$$\Pr[E_{\text{sid}} \mid \neg E_\nu] \leq \frac{q_{\text{sid}}(q_{\text{sid}}-1)}{2^{\eta+1}}. \tag{12}$$

The subsequent reduction argument conditions on $\neg E_\nu \wedge \neg E_{\text{sid}}$.

Step 3: Fix an honest target node and construct an EUF-CMA reduction. Fix any $j \in [N]$ and construct a MAC reduction $\mathcal{B}_j$ that has EUF-CMA oracle access to the target key K_j but does not know K_j itself; for all $i \neq j$ it samples and stores K_i on its own. The reduction simulates the PPA experiment for $\mathcal{A}$ as follows:

- $\mathcal{O}_{\text{Session}}$: honestly generates sessions and records whether a session has been designated as the challenge session;
- $\mathcal{O}_{\text{Corrupt}}(i)$: if $i \neq j$, returns K_i; if $i = j$, aborts the current reduction branch;
- $\mathcal{O}_{\text{Sign}}(i, m)$: if $i \neq j$, returns $\text{HMAC}_{K_i}(m)$; if $i = j$, forwards m to the EUF-CMA signing oracle. Since signing on sid^* is forbidden for honest nodes once the challenge session is created, $\mathcal{B}_j$ never answers a target-key query on the challenge session.

Step 4: Extract a MAC forgery from a successful attack. Let E_j denote the event that $\mathcal{A}$ outputs an acceptable proof P^* and that device j is still honest at the end but did not actually participate in session sid^*. If E_j occurs, verification implies

$$t_j^* = \text{HMAC}_{K_j}(m_j^*) \tag{13}$$

where

$$m_j^* = sid^* \| \text{route_id} \| j \| ID_j \| \mu_j \| u_j^* \| v_j^* \| z_{j-1}^*. \tag{14}$$

Since j did not actually participate in sid^* and honest challenge-session signing queries are forbidden, (m_j^*, t_j^*) cannot come from any historical signing query. $\mathcal{B}_j$ therefore outputs it as a valid EUF-CMA forgery, giving

$$\Pr[E_j] \leq \text{Adv}^{\text{EUF-CMA}}_{\mathcal{B}_j}(\lambda). \tag{15}$$

Step 5: Take a union bound over all possible honest targets. Under $\neg E_\nu \wedge \neg E_{\text{sid}}$, if the adversary wins the k-PPA experiment, there must exist some $j \in \mathcal{U}$ such that E_j occurs. Hence

$$\Pr[E_{\text{fresh}}] \leq \sum_{j \in \mathcal{U}} \Pr[E_j] \leq (N-k) \cdot \max_{j \in \mathcal{U}} \text{Adv}^{\text{EUF-CMA}}_{\mathcal{B}_j}(\lambda). \tag{16}$$

Summing the nonce-collision term, the sid-collision term, and the fresh-forgery term yields eq. (10). □

Theorem 1 gives four direct consequences. First, a single honest node suffices at $k = N - 1$. Second, the bound contains a multiplicative factor of $(N-k)$. Third, reordering or omitting nodes breaks verification because m_i binds the hop index, predecessor state, and both digests. Finally, compromised nodes may still submit semantically incorrect activations; digest-inconsistent transfers across honest-device boundaries are rejected.

4.5 Scope and Extensions

SPAC supports fixed linear routes. DAG merges, asynchronous retries, and MoE routing require additional protocol rules and proofs. Activation digests provide integrity binding, not a privacy guarantee against auxiliary-information inference. Per-hop zero-knowledge proofs could additionally establish $a_i = F_i(\theta_i, a_{i-1})$, but this composition is not implemented. Replacing HMAC with signatures removes device signing keys from the verifier; the Ed25519 substitution is measured in section 5.3. It does not make a compromised verifier enforce acceptance policy or establish runtime integrity without the additional evidence of TA-4.

5 Experimental Evaluation

The evaluation follows the guarantees and encoding contract of sections 3 and 4. It asks four questions:

1) **RQ1: Path authentication.** Does the implementation reject the tested proxy substitutions and changes to the registered route (section 5.2)?
2) **RQ2: Binding mechanisms.** Which session, predecessor, and activation-continuity checks explain the observed attack outcomes (section 5.3)?
3) **RQ3: Encoding behavior.** How do the two forwarding configurations affect compatibility, boundary sensitivity, and within-cell substitutions (section 5.4)?
4) **RQ4: System cost.** What latency, communication, and operating costs accompany these checks (section 5.5)?

Section 5.1 defines the common setup. The negative controls in section 5.6 connect the results to the claim-binding and computational-correctness boundaries of the security model.

5.1 Experimental Setup

Models and topology. The models exercise different boundary-tensor shapes: a **multilayer perceptron (MLP)** uses synthetic vector classification with activations $[1, 64]$–$[1, 128]$ and $\max(6, N)$ feedforward blocks partitioned among the devices; a **Split ResNet-18** variant uses CIFAR-10 (84.17% top-1), with a five-shard stage split (stem $\rightarrow$ stage1 $\rightarrow$ stage2 $\rightarrow$ stage3 $\rightarrow$ head) and boundary tensors $[1, 16, 32, 32] \rightarrow [1, 32, 16, 16] \rightarrow [1, 64, 8, 8] \rightarrow [1, 128, 4, 4] \rightarrow [1, 10]$; and a **ViT** uses CIFAR-10 (62.14% top-1), patch 4, dimension 64, 6 blocks, 4 heads, and token activations $[1, 64, 64] \rightarrow [1, 65, 64]$. MLP and ResNet form the main grid; ViT adds a token-based task. Chain lengths are $N \in \{3, 5, 10\}$. A ViT-base-shaped microbenchmark (input 224×224, hidden dimension 768, 12 blocks) measures cost at larger tensor sizes, separately from task accuracy.

Baselines and attack classes. Baseline A checks only $\|y - y_{\text{ref}}\|_p \leq \delta$; Baseline B authenticates the terminal message $\text{HMAC}_K(sid \| y)$; Baseline C uses independent per-hop HMACs with predecessor-state binding removed; and Baseline E checks device/firmware registration without activation continuity. These controlled constructions correspond to the missing bindings discussed in section 2. SPAC verifies

$P = (y, z_N^{\text{claim}}, \Pi_1, \ldots, \Pi_N)$ using algorithm 1. Activation-injection classes are **AIA-1**, replacement between honest nodes; **AIA-2**, injection across at least one honest boundary; and **AIA-3**, injection following a possibly compromised predecessor with a digest inconsistency exposed downstream. **AIA-4** recomputes a self-consistent sequence within a fully compromised contiguous sub-chain. AIA-1 through AIA-3 are evaluated in section 5.3; AIA-4 is a scope control in section 5.6 (OOS-1/OOS-4).

Metrics and statistics. ASR is output-only acceptance by Baseline A; FAR is SPAC acceptance of an attack submission; AIR is injection acceptance, counted when the injected activation changes the output. QMatch is exact encoding agreement. FRR is the honest-sample failure rate: the denominator is the number n of tested honest samples, and a sample is counted as a failure if verification rejects or, in an encoding-compatibility measurement, if the encoding mismatches. Thus, the combined FRR reported in encoding-stress rows is not a protocol-only rejection rate. The main fixed seed set is $\{42, 52, 62, 72, 82\}$, with $n = 1000$ per configuration for main-grid proxy/topology attacks and $n = 320$ (64 per seed) elsewhere. Optimized within-cell tests use three seeds and 32 inputs per seed. Zero-event cells use the one-sided 95% Clopper–Pearson upper bound $1 - 0.05^{1/n}$: 0.30%, 0.93%, 1.85%, and 7.22% for $n = 1000$, 320, 160, and 40, respectively.

Platform and execution conditions. The shared-LAN platform comprises a 4070TiS PC, a 4090D server, and a DaVinci Mini, using batch size 1 and canonical JSON proof messages over newline-delimited TCP. The cross-host encoding-compatibility grid uses CPU execution, including on the DaVinci Mini. Mixed CUDA/CPU arithmetic and localhost worker-chain tests are identified separately below, so their results are not treated as measurements of a heterogeneous NPU deployment.

Implementation and interpretation. The primary experiments use the canonical SPAC implementation described in section 4. The clause-level grid and additional mixed-arithmetic/within-cell studies use a separate frozen verification implementation with per-record run archives and the split ResNet checkpoint. Accuracy results for this checkpoint are descriptive because checkpoint selection used the test set. The zero-event attack rows check implementation behavior on the covered cases; the MAC assumption and reduction in theorem 1 supply the cryptographic argument. Compatibility, semantic drift, and overhead are measured separately to assess the consequences of the protocol's encoding and trust choices.

5.2 RQ1: Proxy Substitution and Route Tampering

RQ1 tests the session and route bindings of section 4. The full-model substitute executes the benchmark model on one machine and reaches Baseline A ASR 1.0 in every covered configuration. This oracle proxy isolates the limitation of output-only checking (observation 1); it does not measure the cost of extracting a model. Random, distilled, and query-adaptive proxies pass at model- and tolerance-dependent rates (Fig. 3). Route attacks skip, swap, replace, or inject shards. SPAC rejects every covered proxy and route submission in Table 2; the submitted records violate registered identities/claims, ordered bindings, or digest consistency. ResNet and ViT topology spot checks at N=5 ($n = 320$) likewise yield no accepted events. These observations check the implemented acceptance conditions. The distinction between altering a registered claim and executing a different model under the same claim is examined in section 5.6. RQ2 next isolates the checks responsible for the rejection outcomes.

TABLE 2
Output-only Baseline A ASR and SPAC FAR under proxy and topology attacks (n per configuration; seed set $= \{42, 52, 62, 72, 82\}$)

Attack	Model	N	n	ASR (A)	FAR / UB
Proxy family (strongest attack, $\delta = 10^{-3}$)					
full-model proxy	MLP	3,10	1000	1.000	0/1000, 0.003
	ResNet-18	3,10	1000	1.000	0/1000, 0.003
	ViT	5,10	320	1.000	0/320, 0.009
Topology family (MLP, $n_{\text{valid}} = 1000$ per cell)					
skip_node / swap_order	MLP	3,5,10	1000	—	0/1000, 0.003
replace_node / inject_params	MLP	3,5,10	1000	—	0/1000, 0.003

TABLE 3
Representative activation-injection configurations (seed set $= \{42, 52, 62, 72, 82\}$)

Model	Hop / Intensity	n	CRA AIR	SPAC AIR	95% UB
MLP	hop2 / 0.05	320	1.0	0/320	0.009
MLP	hop2 / 0.50	320	1.0	0/320	0.009
ResNet	hop2 / 0.05	320	1.0	0/320	0.009
ResNet	hop3 / 0.05	320	1.0	0/320	0.009
ViT	hop2 / 0.05	320	1.0	0/320	0.009
ViT	hop3 / 0.10	320	1.0	0/320	0.009
ViT	hop4 / 0.20	320	1.0	0/320	0.009

5.3 RQ2: Contribution of the Authentication Bindings

RQ2 separates adjacent digest continuity from predecessor-state binding. On AIA-1 through AIA-3, the state-only CRA-style baseline accepts every covered injection: AIR $= 1.0$ across the three architectures, hops 2–4, and intensities 0.05–0.50. SPAC rejects all covered submissions ($0/320$ per cell, one-sided 95% upper bound 0.93%; Fig. 4(a) and Table 3). These injections expose an encoded-digest inconsistency across an honest boundary, precisely the condition checked by $v_{i-1} = u_i$. The comparison establishes the effect of adding that check to the implemented baseline. Encoding-preserving substitutions require a separate test and are addressed under RQ3.

The intermediate constructions distinguish endpoint authentication from linked hop records. In the covered workloads, terminal-only HMAC (Baseline B) accepts proxy substitution, order changes, and injection. Independent per-hop HMAC (Baseline C) accepts 38% of order swaps and 72% of injections on the tested MLP configuration. These rates concern the specific message bindings and verifier rules of the baselines, rather than a general limitation of HMAC.

The clause-level grid then varies the verifier directly: the full scheme, removal of $v_{i-1} = u_i$, and independent per-hop authentication without predecessor binding. Across $5{,}184$ enumerated submissions (96 per cell, with inputs and seeds reused across cells), the fractions describe those exact grids (Fig. 4(b)). Removing continuity accepts all $288/288$ altered-payload submissions; retaining it rejects all $288/288$. Independent authentication accepts reverse-order re-signing

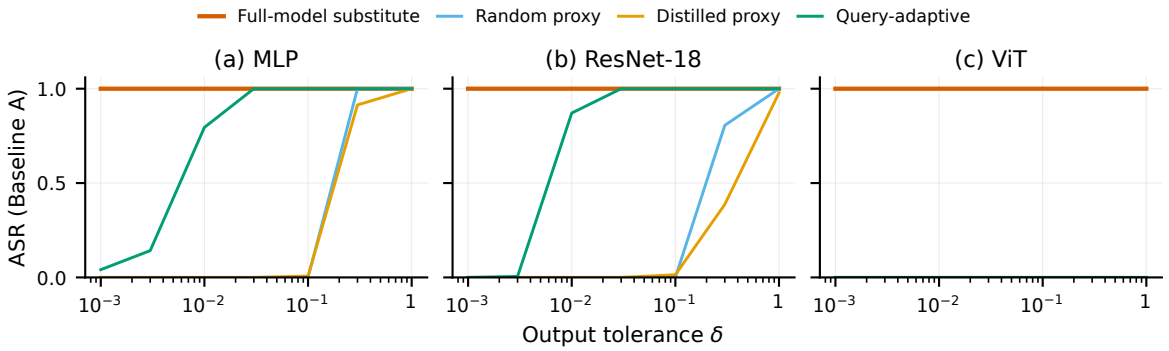


Fig. 3. ASR–δ curves of the four proxy attacks (N=5; five seeds). The full-model substitute (vermillion) stays at ASR= 1 at every tolerance and under every architecture, while the weaker proxies (light blue: random; orange: distilled; green: query-adaptive) become acceptable only as δ is relaxed, at thresholds that depend on the model. SPAC rejects all of them at every δ (FAR= 0 in all 252 measured cells).

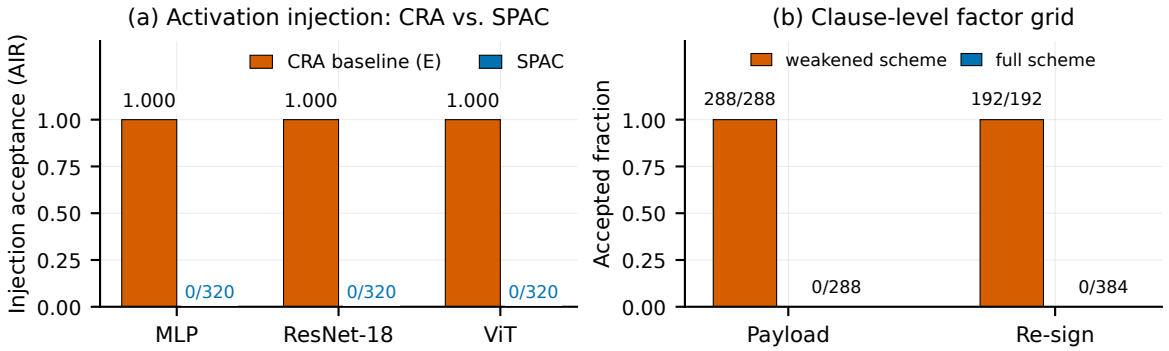


Fig. 4. The continuity binding versus weaker verification. (a) Activation-injection acceptance of the CRA-style baseline and SPAC, across all covered configurations per architecture (4 MLP, 9 ResNet-18, and 9 ViT configurations, $n = 320$ each; labels show per-cell counts, with zero SPAC events in every cell and one-sided 95% upper bound 0.93% per cell). (b) Clause-level factor grid over $5{,}184$ submissions (red: the weakened verification surface in each contrast; blue: the full scheme): without the continuity clause all altered payloads pass; with it none do; per-hop independent authentication accepts reverse-order re-signing that all chained constructions reject. The one scenario that every configuration accepts by design—adversarial computation with consistent declarations—is not shown here; section 5.6 examines this boundary.

on $192/192$ submissions, while chained constructions accept $0/384$. Cross-session transplants, route splices, tag reuse, and tested same-nonce submissions are rejected throughout. This cross-session result does not provide single-use semantics for an unchanged proof under the same unexpired context. Self-consistent adversarial computation is accepted by every configuration, as expected from the scope in section 3.

Table 4 connects these outcomes to the proof fields. Removing μ_i accepts inject_params submissions (acceptance 1.0), exposing registered-claim inconsistency. Removing sid/ν permits cross-session replay; removing u_i/v_i removes activation-continuity checks; and removing z_i accepts tested swap_order submissions (acceptance 1.0). These are distinct roles in the construction, while actual runtime model integrity remains dependent on TA-4. Substituting Ed25519 preserves benign functionality and increases latency by 1.58–$1.71\times$, measuring the public-signature option discussed in section 4.5.

Selected collusion patterns check the implementation under the corruption model of theorem 1. Across 324 covered $(model, N, k, corruption\ pattern, attack)$ groups ($n = 320$ each over five seeds; three model families, chain lengths 3/5/10, key-forgery and node-skipping attacks, and head/middle/tail/non-contiguous layouts), no forgery succeeds (one-sided 95% upper bound 0.93% per group). The grid samples concrete patterns; the theorem addresses fresh false attribution under its stated adaptive-corruption game. With the binding checks identified, RQ3 examines the encoding equivalence classes to which those checks apply.

TABLE 4
Component ablation (MLP, seed set $= \{42, 52, 62, 72, 82\}$). The last row reports benign acceptance; the other rows report attack acceptance.

Variant	Attack	n	SPAC	Ablation	Explanation
w/o μ_i	inject_params	320	0/320	1.0	claim binding failure
w/o sid/ν	replay (cross-sess.)	320	0/320	1.0	freshness failure
w/o u_i/v_i	AIA (s=0.2)	320	0/320	1.0	digest chain failure
w/o z_i	swap_order	320	0/320	1.0	chain-state failure
HMAC→Ed25519	honest inference	320	—	1.0	latency ×1.58–1.71

5.4 RQ3: Encoding Compatibility and Residual Risk

RQ3 follows the two contracts in Table 1. Under the tested shared numeric configuration, QMatch is 1.0 across FP16/INT8/mixed precision and reported FRR is $0/320$ (upper bound 0.93%; Table 5). This CPU-mode grid checks agreement under nearly identical numerics. Mixed arithmetic, boundary stress, and within-cell optimization then examine three different ways that transfer encoding can affect compatibility or semantics.

Mixed arithmetic. The split ResNet chain uses CUDA fp32 → CPU fp32 → CPU fp16. Protocol forwarding accepts $300/300$ legal inputs under both the integer-code (all-W1)

TABLE 5
Quantization stability under the shared numeric configuration ($n = 320$ per configuration, seed set $= \{42, 52, 62, 72, 82\}$). The FP16, INT8, and Mixed columns report QMatch.

Model	QMatch (FP16)	QMatch (INT8)	QMatch (Mixed)	FRR	FRR 95% UB
MLP	1.0	1.0	1.0	0/320	0.009
ResNet-18	1.0	1.0	1.0	0/320	0.009

and default float-forwarding (W2) configurations. For the all-W1 mixed-stack readout on the stratified 1,000-input subset, *Plain* denotes the paired CPU-fp32 reference execution; all-W1 matches Plain's accuracy, and its predicted labels agree with the Plain predictions on 99.10% of inputs. Independently re-quantizing consumer tensors matches the producer's codes on 1.56%, 22.66%, and 92.97% of the 128 measurements at the three edges (Fig. 5(a)). These measurements concern separate operations. Code forwarding verifies received bytes before decoding, so independent requantization is diagnostic. Float forwarding must still match the registered encodings of the actual transferred tensor (eq. (6)); its acceptance result applies to the tested placement of encoding and conversion. The diagnostic mismatch rates do not replace that acceptance check.

Boundary sensitivity. The local stress test perturbs only the 0.02% of coordinates closest to a quantization boundary, with per-coordinate budget 0.1 of the local half-cell width. Attack acceptance remains FAR $= 0/320$ on both models. Reported honest-sample FRR for near/mid/far groups is $0.479/0.434/0.348$ on MLP and $0.474/0.498/0.469$ on ResNet. The 0.35–0.50 range measures sensitivity under this stress model, using the combined failure statistic defined in section 5.1. It is not an estimate of protocol-only rejection on ordinary heterogeneous traffic.

Within-cell float substitution. A different failure mode preserves $E_a(a') = E_a(a)$ while changing downstream computation. A fixed-cell drift-maximization attack exceeds the 5% relative-drift threshold on $0/320$ MLP samples (with $2/320$ label flips below the threshold) and $6/320$ ResNet samples (maximum $\|\Delta y\|_\infty = 0.005813$, relative drift 0.0669). An attack is counted as harmful when verification accepts, the clean prediction is correct, and the attacked prediction is incorrect. Gradient optimization targets this event on 96 clean-correct inputs per budget, shared across arms. At the first edge, ten steps yield $67/96$ harmful events (69.8%); 50 or 200 steps yield $69/96$ (71.9%). The second edge yields $19/96$ and $20/96$ (19.8%, 20.8%). Identity, zero-step, radius-matched random controls, and the final-edge attack at 200 steps each yield $0/96$ (Fig. 5(b)). The early-edge results demonstrate why equality of a lossy encoding cannot be interpreted as semantic equivalence. The contrast with radius-matched random controls shows why untargeted perturbation outcomes do not characterize an adversary optimizing within one encoding cell. These accepted same-code cases complement the rejected digest-inconsistent injections under RQ2: the verifier applies the same equality check, but the attacker has preserved the object that the check authenticates.

Changing the authenticated wire object. Transmitted-code authentication removes this intermediate-link substitution freedom by hashing the bytes that are received and decoded. On the corresponding inputs, the configuration accepts and preserves correctness on $96/96$ at every edge. Accuracy on the full 10,000-image test set decreases from 84.17% to 84.16%, an observed accuracy difference of 0.01 pp for the selected checkpoint; all sessions verify. Under INT8 and N=5, the tests therefore expose harmful float substitutions up to 71.9% in the optimized early-edge cases and evaluate a code-forwarding mitigation with an observed 0.01 pp accuracy difference. The structural mitigation concerns authenticated internal byte objects, while returned raw outputs remain bound through their registered encoding. The accuracy and stress statistics retain their model- and configuration-specific interpretation. This choice also changes the downstream numeric input: float forwarding retains the received tensor, whereas code forwarding computes on its decoding. The accuracy comparison evaluates that numerical change on the selected checkpoint. The float-mode attack rate and code-mode mitigation therefore describe two different transfer contracts.

TABLE 6
System overhead (seed set $= \{42, 52, 62, 72, 82\}$, controlled environment, batch size $= 1$). The ΔT CI95 column reports the 95% confidence-interval half-width.

Model	N	T_{plain} (ms)	T_{auth} (ms)	ΔT (ms)	ΔT CI95	Thr. (sess/s)	ΔB (B)	$\lvert P\rvert$ (B)
MLP	3	0.55	1.20	0.65	0.25	833	384	1 098
MLP	5	0.65	1.65	1.00	0.24	606	640	1 760
MLP	10	0.95	2.79	1.84	0.24	358	1 280	3 417
ResNet-18	3	3.13	3.99	0.85	0.10	251	384	1 098
ResNet-18	5	3.27	4.92	1.65	0.48	203	640	1 760
ResNet-18	10	3.42	5.98	2.56	0.35	167	1 280	3 417

Relation to the encoding analysis. The reference in remark 1 concerns encoding agreement under simplifying assumptions. For FP16/INT8/mixed precision, $\text{Bound}_{\text{theory}} = 1.0$ with $\Delta_{\text{QMatch}} = 0.0$. Here $\Delta_{\text{QMatch}} = \text{QMatch}_{\text{measured}} - \text{Bound}_{\text{theory}}$, so its sign indicates whether the measured rate is below or above the reference. The SA-1 phase deviations (5.8×10^{-4}–6.2×10^{-2}) and SA-2 channel correlations (0.167–0.235) show that the assumptions do not hold generally. Under INT4/per-tensor/reduced-range/ϵ-sweep stress, $71/72$ non-trivial rows satisfy QMatch $\geq \text{Bound}_{\text{theory}}$, with $\Delta_{\text{QMatch}} \in [-0.38, +0.77]$. Deployment conclusions consequently use measured forwarding acceptance and encoding behavior, with cost assessed next under RQ4.

5.5 RQ4: Authentication and Operating Costs

RQ4 measures the price of the checks isolated under RQ2 and the encoding choices examined under RQ3. Table 6 reports the primary grid: plain latency is 0.55–0.95 ms for MLP and 3.13–3.42 ms for ResNet; authenticated latency is 1.20–2.79 ms and 3.99–5.98 ms, respectively. The added 0.65–2.56 ms grows approximately with N. At batch size 1, the authenticated single-session rate is 358–833 sessions/s for MLP and 167–251 for ResNet. Additional communication volume ΔB per session grows from 384 B at N=3 to $1{,}280$ B at N=10, and proof size from $1{,}098$ B to $3{,}417$ B. This observed scaling agrees with the $O(N\lambda)$ metadata construction; absolute latency should be read alongside the plain baseline.

Cost components and larger activations. On the cross-host chain, median HMAC cost is 5.4–$6.8\,\mu$s per hop; median complete protocol cost peaks at 0.136 ms over 500 iterations. INT8 encoding costs 45–$59\,\mu$s per hop, compared

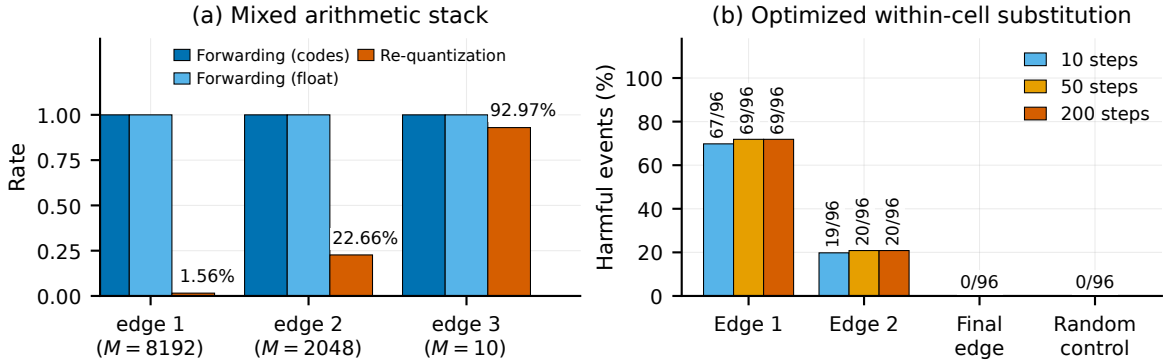


Fig. 5. The activation encoder under heterogeneous arithmetic and adversarial substitution (split ResNet-18). (a) Mixed stack (CUDA fp32→CPU fp32→CPU fp16): protocol forwarding accepts $300/300$ legal inputs under both configurations, while independently re-quantized tensors match the producer's codes exactly in only $1.56\%/22.66\%/92.97\%$ of measurements at the three edges ($n = 128$). (b) Optimized within-cell substitution: harmful events by gradient budget at the first two edges (96 clean-correct inputs, 32 per seed, three seeds, shared across arms); the final edge and all controls yield $0/96$.

with about $3.6\,\mu$s for FP16. On ViT/CIFAR-10, plain latency is 5.23 ms (N=5) and 5.17 ms (N=10), versus authenticated 6.08 ms and 6.87 ms: increments of 0.85 ms and 1.71 ms, with rates of about 169 and 150 sessions/s. Encoding takes $0.41/0.73$ ms per session and the post-encoding boundary payload is about 4.28 KB. The ViT-base-shaped microbenchmark at N=5 measures 85.53 ms plain and 89.62 ms authenticated, adding 4.09 ms with about 11.16 sessions/s, 1.90 ms total encoding, and a boundary payload of about 151 KB. These larger-shape measurements characterize scaling pressure independently of task accuracy.

Operating conditions. A localhost five-hop MLP worker chain examines session overlap, failure, and route changes. At concurrency $1/2/4$, all covered sessions are accepted over five seeds; average per-session latency is $6.59/6.12/14.01$ ms and aggregate throughput about $177/291/235$ sessions/s. Worker termination before execution is detected in every covered trial, and a restarted full-session retry succeeds. Permuting physical workers while retaining the registered logical route is rejected throughout. These transport tests complement the primary latency grid, while WAN retransmission and heterogeneous NPU operation remain outside the measured conditions.

5.6 Trust-Boundary Controls and Interpretation

The preceding questions evaluate authentication behavior, encoding risk, and cost. Two negative controls now test the boundary between an accepted participation transcript and stronger statements about the computation that produced it, as defined in section 3.

Self-consistent compromised computation. In AIA-4, compromised devices in a contiguous sub-chain recompute consistent $u_i/v_i/t_i$ records using their own keys. Such executions are accepted, as are the consistent-declaration cases in the clause-level grid. This is the expected consequence of authenticating records rather than verifying $a_i = F_i(\theta_i, a_{i-1})$ for compromised hops. It complements RQ1 and RQ2: rejection requires a violated authentication or consistency condition, while self-consistent semantic deviations require an additional correctness mechanism (section 4.5).

Registered claims and hardware endorsement. Under TA-3, a keyed device can run a different model or code image while reporting the registered μ_i; the valid declaration is accepted. A simulated trusted-measurement component instead checks the local measurement before signing and rejects a mismatch, illustrating the intended TA-4 distinction. This control clarifies the interpretation of claim-binding ablation results; it does not evaluate a hardware-attestation implementation. The measured deployment envelope therefore combines fresh session records and encoded transfer checks with the stated key-protection assumptions. A deployment requiring hardware-endorsed measurements or computational correctness must supply that additional evidence alongside SPAC.

6 Conclusion

SPAC authenticates session-bound path records for linear-chain distributed inference. Its proof binds identities, registered shard claims, record order, and adjacent encoded-activation digests. The reduction in theorem 1 bounds fresh false attribution of honest participation under the specified $k < N$ corruption game. The claim-binding baseline requires protected verifier keys and secret honest-device keys, but no device-side hardware attestation; hardware-endorsed measurements and computational correctness require additional assurance.

The tested proxy, topology, and digest-inconsistent injection workloads produced no accepted forgeries, while weakened verification variants accepted specific constructed tampering. These observations cover the tested cases and do not imply that all adversarial activations are rejected: optimized within-cell float substitutions produced harmful outcomes on $69/96$ early-edge samples. Transmitted-code authentication removes that substitution mechanism on tested links, with an observed accuracy difference of 0.01 pp on the selected ResNet checkpoint. Added latency was 0.65–2.56 ms on the primary MLP/ResNet grid, with proof metadata linear in chain length.

Limitations and Future Work. Compromised nodes can submit semantically incorrect but self-consistent records, and returned outputs are bound at encoding granularity. The experiments used a simulated trusted-measurement

component to illustrate hardware endorsement; a real hardware-attestation implementation remains future work. The cross-host chain uses the DaVinci Mini in CPU mode; mixed arithmetic was evaluated on one host. Model coverage includes ViT/CIFAR-10 and a larger-shape microbenchmark, but no pretrained NLP models or NPU-backend evaluation. Future work includes broader hardware validation, adaptive collusion experiments, general topologies (section 4.5), and composition with computational-correctness mechanisms.

REFERENCES

[1] S. Teerapittayanon, B. McDanel, and H. T. Kung, "Distributed deep neural networks over the cloud, the edge and end devices," in *IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2017, pp. 328–339.

[2] E. Li, L. Zeng, Z. Zhou, and X. Chen, "Edge AI: On-demand accelerating deep neural network inference via edge computing," *IEEE Transactions on Wireless Communications*, vol. 19, no. 1, pp. 447–457, 2020.

[3] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. A. y Arcas, "Communication-efficient learning of deep networks from decentralized data," *Proceedings of the 20th International Conference on Artificial Intelligence and Statistics (AISTATS)*, pp. 1273–1282, 2017.

[4] D. Crankshaw, X. Wang, G. Zhou, M. J. Franklin, J. E. Gonzalez, and I. Stoica, "Clipper: A low-latency online prediction serving system," in *USENIX Symposium on Networked Systems Design and Implementation (NSDI)*, 2017, pp. 613–627.

[5] N. Carlini, M. Jagielski, and I. Mironov, "Cryptanalytic extraction of neural network models," in *Advances in Cryptology – CRYPTO*, 2020.

[6] A. Shamir, I. Canales-Martinez, A. Hambitzer, J. Chávez-Saab, F. Rodríguez-Henríquez, and N. Satpute, "Polynomial time cryptanalytic extraction of neural network models," arXiv:2310.08708, 2023.

[7] M. Ambrosin, M. Conti, A. Ibrahim, G. Neven, A. Sadeghi, and M. Schunter, "SANA: Secure and scalable aggregate network attestation," in *Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2016.

[8] A. Ahmadi *et al.*, "CATT: A comprehensive framework for collective remote attestation," *ACM Computing Surveys*, 2024.

[9] Y. Chen, X. Dong, J. Guo, Y. Shen, A. Wang, and X. Wang, "Hard-label cryptanalytic extraction of neural network models," arXiv:2409.11646, 2024, preprint.

[10] N. Carlini, J. Chávez-Saab, A. Hambitzer, F. Rodríguez-Henríquez, and A. Shamir, "Polynomial time cryptanalytic extraction of deep neural networks in the hard-label setting," arXiv:2410.05750, 2024, preprint.

[11] S. A. Crosby and D. S. Wallach, "Efficient data structures for tamper-evident logging," in *Proceedings of the 18th USENIX Security Symposium*, 2009, pp. 317–334.

[12] A. Lysyanskaya, S. Micali, L. Reyzin, and H. Shacham, "Sequential aggregate signatures from trapdoor permutations," in *Advances in Cryptology – EUROCRYPT*, 2004, pp. 74–90.

[13] B. Kuang, A. Fu, S. Yu, G. Yang, M. Su, and Y. Zhang, "ESDRA: An efficient and secure distributed remote attestation scheme for IoT swarms," *IEEE Internet of Things Journal*, vol. 6, no. 5, pp. 8372–8383, 2019.

[14] A. Hofmann *et al.*, "PIRANHAS: Privacy-preserving remote attestation for non-hierarchical asynchronous groups," IACR ePrint 2025/2228, 2025.

[15] M. Conti, N. Dragoni, T. Frassetto, and A. Sadeghi, "RADIS: Remote attestation of distributed IoT services," in *IEEE International Conference on Distributed Computing Systems (ICDCS)*, 2018, pp. 748–758.

[16] T. A. Abera, N. Asokan, L. Davi, J. Ekberg, T. Nyman, A. Paverd, A. Sadeghi, and G. Tsudik, "C-FLAT: Control-flow attestation for embedded systems software," in *ACM Conference on Computer and Communications Security (CCS)*, 2016.

[17] G. Dessouky, S. Zeitouni, T. Nyman, A. Paverd, L. Davi, P. Koeberl, N. Asokan, and A. Sadeghi, "LO-FAT: Low-overhead control flow attestation in hardware," in *Proceedings of the 54th Annual Design Automation Conference (DAC)*, 2017.

[18] T. Caulfield *et al.*, "SpecCFA: Speculative control-flow attestation," in *Annual Computer Security Applications Conference (ACSAC)*, 2024.

[19] ——, "TRACES: Tee-based runtime auditing for commercial embedded systems," in *Annual Computer Security Applications Conference (ACSAC)*, 2024.

[20] R. M. Halldórsson, E. Dushku, and N. Dragoni, "ARCADIS: Asynchronous remote control-flow attestation of distributed IoT services," *IEEE Access*, vol. 9, pp. 144 880–144 894, 2021.

[21] B. Chen, S. Waiwitlikhit, I. Stoica, and D. Kang, "ZKML: An optimizing system for ML inference in zero-knowledge proofs," in *European Conference on Computer Systems (EuroSys)*, 2024.

[22] H. Lycklama, A. Viand, N. Avramov *et al.*, "Artemis: Efficient commit-and-prove SNARKs for zkML," arXiv:2409.12055, 2024, preprint.

[23] H. Sun, J. Li, and H. Zhang, "zkLLM: Zero knowledge proofs for large language models," arXiv:2404.16109, 2024, preprint.

[24] Y. Sun, Y. Li, Y. Zhang, Y. Jin, and H. Zhang, "SVIP: Towards verifiable inference of open-source large language models," arXiv:2410.22307, 2024, preprint.

[25] K. Wang, F. Qu, L. Xia, Z. Zhao, C. Tong, L. Ai, and E. Yang, "VeriLLM: A lightweight framework for publicly verifiable decentralized inference," arXiv:2509.24257, 2025, preprint.

[26] O. Baser, E. Sadeghi, E. Wang, D. R. Alves, S. Kazemian, H. Kang, S. P. Chinchali, and S. Vishwanath, "TensorCommitments: A lightweight verifiable inference for language models," arXiv:2602.12630, 2026, preprint.

[27] P. Anchuri, M. Campanelli, P. Cesaretti, R. Gennaro, T. M. Jois, H. S. Kayman, and T. Ozdemir, "Towards verifiable AI with lightweight cryptographic proofs of inference," arXiv:2603.19025, 2026, preprint.

[28] M. Chrapek, M. Copik, E. Mettaz, and T. Hoefler, "Confidential LLM inference: Performance and cost across CPU and GPU TEEs," arXiv:2509.18886, 2025, preprint.

[29] C. Kocaoğullar, G. Petri, D. P. Mulligan, D. Miller, H. J. M. Vincent, S. Xiong, and A. R. Beresford, "Careful whisper: Attestation for peer-to-peer confidential computing networks," arXiv:2507.14796, 2025, preprint.

[30] A. Galanou, F. Lubitz, H. Jeon, C. Fetzer, and R. Kapitza, "Full trust alchemist: Reforging attestation for cloud-based confidential workloads," in *ACM Middleware Conference*, 2025.

[31] L. Martinico and M. Kohlweiss, "AGATE: Augmented global attested trusted execution in the universal composability framework," in *IEEE Computer Security Foundations Symposium (CSF)*, 2025.

[32] Z. He, M. Wang, X. Gao, Y. Luo, L. Liu, and S. Fu, "ENSI: Efficient non-interactive secure inference for large language models," arXiv:2509.09424, 2025, preprint.

[33] Y. Zhu, C. Jin, K. M. M. Aung, and X. Xiao, "EncFormer: Secure and efficient transformer inference over encrypted data," arXiv:2604.09975, 2026, preprint.